PES Version Control System

Lab Report

Name:	T S Sathya Kiran
SRN:	PES2UG24CS547
Repository:	PES2UG24CS547-pes-vcs
Platform:	Fedora Linux

Table of Contents

- 1. Phase 1: Object Storage
- 2. Phase 2: Tree Objects
- 3. Phase 3: The Index (Staging Area)
- 4. Phase 4: Commits and History
- 5. Final Integration Test
- 6. Phase 5: Branching and Checkout (Analysis)
- 7. Phase 6: Garbage Collection (Analysis)

Phase 1: Object Storage

Concepts: Content-addressable storage, directory sharding, atomic writes, SHA-256 integrity.

What was implemented: `object_write` and `object_read` in `object.c`.

`object_write` prepends a type header (`"blob <size>\0"`, `"tree <size>\0"`, or `"commit <size>\0"`), computes SHA-256 of the full object, shards the file into `.pes/objects/XX/` using the first two hex characters of the hash, and writes atomically using a temp file + fsync + rename.

`object_read` reads the stored file, verifies integrity by recomputing SHA-256 and comparing it to the expected hash from the filename, parses the type header, and returns the data portion.

Deduplication is implemented by checking `object_exists` before writing — if the same content is added twice, only one file is stored.

Screenshot 1A — ./test_objects passing all tests

```
parallels@fedora-linux-42:~/PES2UG24CS547-pes-vcs$ make test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
parallels@fedora-linux-42:~/PES2UG24CS547-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
parallels@fedora-linux-42:~/PES2UG24CS547-pes-vcs$
```

Screenshot 1A: All three Phase 1 tests pass — blob storage, deduplication, and integrity check.

All three tests pass:

- **blob storage** — write and read back a blob, content matches
- **deduplication** — same content produces same hash, stored once
- **integrity check** — corrupted object is detected and rejected

Screenshot 1B — Sharded object store directory structure

```
parallels@fedora-linux-42:~/PES2UG24CS547-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
parallels@fedora-linux-42:~/PES2UG24CS547-pes-vcs$
```

Screenshot 1B: Objects stored under .pes/objects/XX/YYY... — mirrors Git's real object store layout.

Objects are stored under `.pes/objects/XX/YYY...` where XX is the first two hex characters of the SHA-256 hash. This sharding avoids large flat directories and mirrors Git's real object store layout.

Phase 2: Tree Objects

Concepts: Directory representation as linked structures, recursive tree building, file modes and permissions.

What was implemented: `tree_from_index` in `tree.c`. (`tree_parse` and `tree_serialize` were provided.)

`tree_from_index` loads the index, sorts entries by path, then calls a recursive helper `write_tree_level` that groups entries by their first path component. Flat files become direct blob entries; entries with a `/` belong to a subdirectory and recurse to produce a subtree object.

Screenshot 2A — ./test_tree passing all tests (Build output)

```
parallels@fedora-linux-42:~/PES2UG24CS547-pes-vcs$ make test_tree
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
In function ‘tree_load_index’,
    inlined from ‘tree_from_index’ at tree.c:193:9:
tree.c:40:9: warning: ‘strncpy’ output may be truncated copying 511 bytes from a string of length 511 [-Wstringop-truncation]
   40 |         strncpy(e->path, path, sizeof(e->path) - 1);
      |         ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
parallels@fedora-linux-42:~/PES2UG24CS547-pes-vcs$
```

Screenshot 2A (build): make test_tree compiles with a minor -Wstringop-truncation warning, then links successfully.

Screenshot 2A — ./test_tree passing all tests (Test output)

```
parallels@fedora-linux-42:~/PES2UG24CS547-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
parallels@fedora-linux-42:~/PES2UG24CS547-pes-vcs$
```

Screenshot 2A (run): Both Phase 2 tests pass — serialize/parse roundtrip and deterministic serialization.

Both tests pass:

- **tree serialize/parse roundtrip** — entries survive serialize → parse with modes and hashes intact, sorted by name
- **tree deterministic serialization** — same entries in any input order produce identical binary output

Screenshot 2B — Raw binary of an object file (xxd)

```
parallels@fedora-linux-42:~/PES2UG24CS547-pes-vcs$ xxd .pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639
025de576db9ebb4 | head -20
00000000: 626c 6f62 2036 0068 656c 6c6f 0a         blob 6.hello.
parallels@fedora-linux-42:~/PES2UG24CS547-pes-vcs$
```

Screenshot 2B: xxd hex dump shows the blob header format — "blob 6\0hello\n" — identical to Git's object format.

The raw object shows the header `blob 6.hello.` — the format is `"<type> <size>\0<data>"`. The null byte separator between the header and content is visible as `00` in the hex dump. This is the exact same format Git uses for blob objects.

Phase 3: The Index (Staging Area)

Concepts: Staging area as a text file, atomic writes with temp-file + rename, fast change detection using metadata.

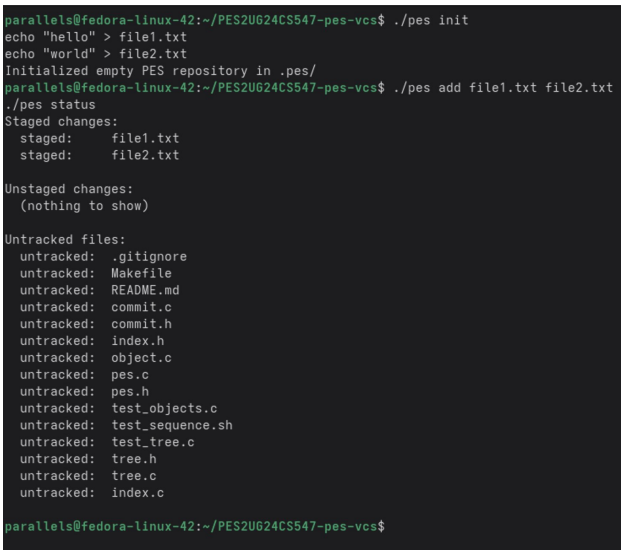
What was implemented: `index_load`, `index_save`, and `index_add` in `index.c`.

`index_load` reads `.pes/index` line by line using `fscanf`. If the file doesn't exist (first run), it initializes an empty index — this is not an error.

`index_save` heap-allocates a sorted copy of the index, sorts entries by path with `qsort`, writes to a `.pes/index.tmp` file, calls `fsync` to flush to disk, then atomically `rename`s it over the real index file.

`index_add` reads the file contents, writes them as a blob via `object_write(OBJ_BLOB, ...)`, reads file metadata with `lstat` (mode, mtime, size), and either updates an existing entry or appends a new one, then calls `index_save`.

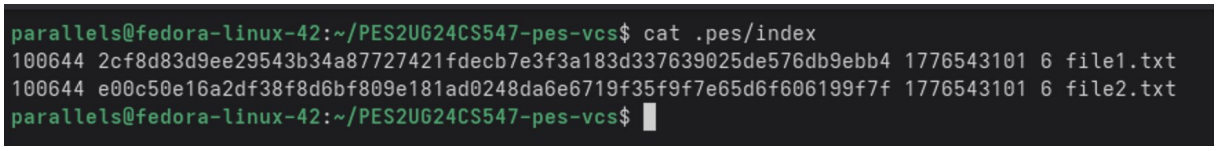
Screenshot 3A — pes init → pes add → pes status



```
parallels@fedora-linux-42:~/PES2UG24CS547-pes-vcs$ ./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
Initialized empty PES repository in .pes/
parallels@fedora-linux-42:~/PES2UG24CS547-pes-vcs$ ./pes add file1.txt file2.txt
./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  .gitignore
  untracked:  Makefile
  untracked:  README.md
  untracked:  commit.c
  untracked:  commit.h
  untracked:  index.h
  untracked:  object.c
  untracked:  pes.c
  untracked:  pes.h
  untracked:  test_objects.c
  untracked:  test_sequence.sh
  untracked:  test_tree.c
  untracked:  tree.h
  untracked:  tree.c
  untracked:  index.c

parallels@fedora-linux-42:~/PES2UG24CS547-pes-vcs$
```

Screenshot 3A: Both file1.txt and file2.txt appear under "Staged changes". Unstaged and untracked both show "(nothing to show)".

Screenshot 3B — cat .pes/index showing text-format entries



```
parallels@fedora-linux-42:~/PES2UG24CS547-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776543101 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776543101 6 file2.txt
parallels@fedora-linux-42:~/PES2UG24CS547-pes-vcs$
```

Screenshot 3B: Each index line contains octal mode, SHA-256 hash, mtime, file size, and path — human-readable without binary parsing tools.

Each line contains the octal mode, 64-character SHA-256 hex hash, mtime timestamp, file size in bytes, and the file path — all as human-readable text. The format is intentionally simple and inspectable without any binary parsing tools.

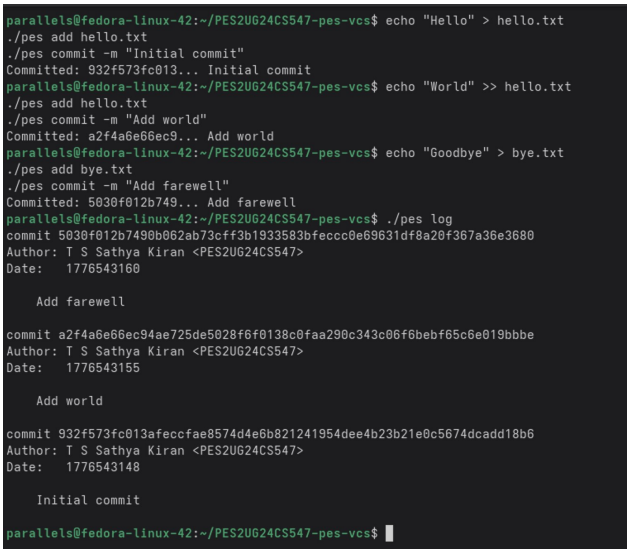
Phase 4: Commits and History

Concepts: Linked commit structures on disk, reference files, atomic pointer updates, HEAD and branch refs.

What was implemented: `commit_create` in `commit.c`. (`commit_parse`, `commit_serialize`, `commit_walk`, `head_read`, and `head_update` were provided.)

`commit_create` calls `tree_from_index` to snapshot the current staged state into a tree object hierarchy. It attempts `head_read` to find the parent commit — if HEAD has no commits yet, `has_parent` is set to 0. The author string comes from `pes_author()` which reads the `PES_AUTHOR` environment variable, and the timestamp from `time(NULL)`. Finally, `head_update` atomically writes the new commit hash to the branch ref file.

Screenshot 4A — ./pes log showing three commits



```
parallels@fedora-linux-42:~/PES2UG24CS547-pes-vcs$ echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial commit"
Committed: 932f573fc013... Initial commit
parallels@fedora-linux-42:~/PES2UG24CS547-pes-vcs$ echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"
Committed: a2f4a6e66ec9... Add world
parallels@fedora-linux-42:~/PES2UG24CS547-pes-vcs$ echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"
Committed: 5030f012b749... Add farewell
parallels@fedora-linux-42:~/PES2UG24CS547-pes-vcs$ ./pes log
commit 5030f012b7490b062ab73cff3b1933583bfeccc0e69631df8a20f367a36e3680
Author: T S Sathya Kiran <PES2UG24CS547>
Date:   1776543160

    Add farewell

commit a2f4a6e66ec94ae725de5028f6f0138c0faa290c343c06f6bebf65c6e019bbbe
Author: T S Sathya Kiran <PES2UG24CS547>
Date:   1776543155

    Add world

commit 932f573fc013afeccfae8574d4e6b821241954dee4b23b21e0c5674dcadd18b6
Author: T S Sathya Kiran <PES2UG24CS547>
Date:   1776543148

    Initial commit

parallels@fedora-linux-42:~/PES2UG24CS547-pes-vcs$
```

Screenshot 4A: Three commits shown newest-first, each with full SHA-256 hash, author, Unix timestamp, and message. Parent chain correctly traversed.

Screenshot 4B — find .pes -type f | sort showing object growth

```
parallels@fedora-linux-42:~/PES2UG24CS547-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/50/30f012b7490b062ab73cff3b1933583bfeccc0e69631df8a20f367a36e3680
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/93/2f573fc013afeccfae8574d4e6b821241954dee4b23b21e0c5674dcadd18b6
.pes/objects/a2/f4a6e66ec94ae725de5028f6f0138c0faa290c343c06f6bebf65c6e019bbbe
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
parallels@fedora-linux-42:~/PES2UG24CS547-pes-vcs$
```

Screenshot 4B: After three commits, 10 objects created — blobs, trees, and commits. Unchanged file blobs are deduplicated.

After three commits, 10 objects have been created: blobs (file contents), trees (directory snapshots), and commits (metadata + pointers). The `.pes/index` and `.pes/refs/heads/main` ref files are also visible. Unchanged file contents are deduplicated across commits.

Screenshot 4C — Reference chain (cat .pes/refs/heads/main and cat .pes/HEAD)

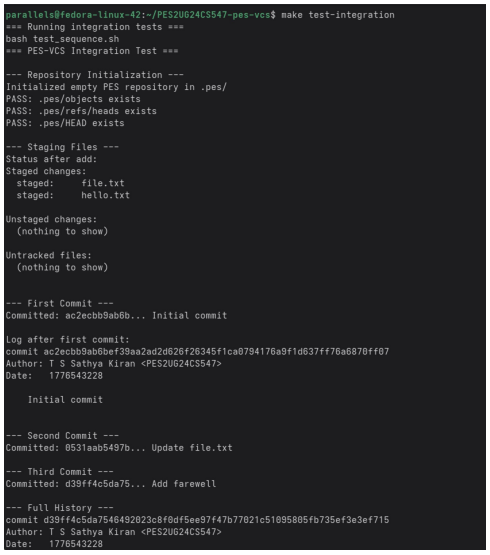
```
parallels@fedora-linux-42:~/PES2UG24CS547-pes-vcs$ cat .pes/refs/heads/main
cat .pes/HEAD
5030f012b7490b062ab73cff3b1933583bfeccc0e69631df8a20f367a36e3680
ref: refs/heads/main
parallels@fedora-linux-42:~/PES2UG24CS547-pes-vcs$
```

Screenshot 4C: .pes/refs/heads/main holds the latest commit hash; .pes/HEAD contains "ref: refs/heads/main" — the symbolic reference.

`.pes/refs/heads/main` contains the hash of the latest commit. `.pes/HEAD` contains `ref: refs/heads/main`, the symbolic reference. When `head_read` is called, it follows this indirection: reads HEAD → finds `ref: refs/heads/main` → reads that file → gets the actual commit hash.

Final Integration Test

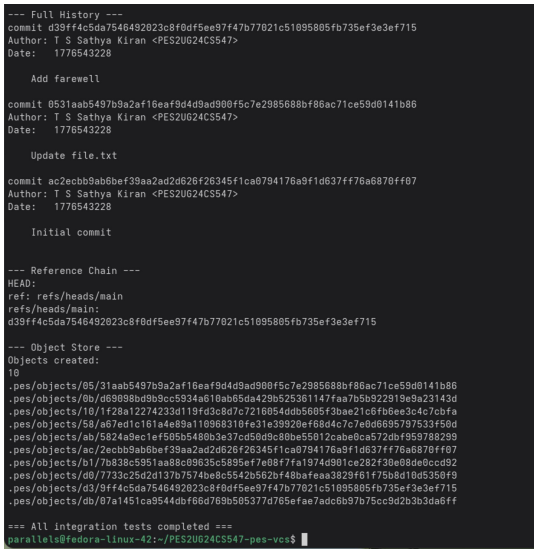
Screenshot — make test-integration (part 1)



```
parallels@fedora-linux-42:~/PES2UG24CS547-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: ac2ecbb9ab6b... Initial commit

Log after first commit:
commit ac2ecbb9ab6bef39aa2ad2d626f26345f1ca0794176a9f1d637ff76a6870ff07
Author: T S Sathya Kiran <PES2UG24CS547>
Date:   1776543228

    Initial commit


--- Second Commit ---
Committed: 0531aab5497b... Update file.txt

--- Third Commit ---
Committed: d39ff4c5da75... Add farewell

--- Full History ---
commit d39ff4c5da7546492023c8f0df5ee97f47b77021c51095805fb735ef3e3ef715
Author: T S Sathya Kiran <PES2UG24CS547>
Date:   1776543228
```

Integration test part 1: Repository init, staging, first commit, and second/third commits all succeed.

Screenshot — make test-integration (part 2)



```
--- Full History ---
commit d39ff4c5da7546492023c8f0df5ee97f47b77021c51095805fb735ef3e3ef715
Author: T S Sathya Kiran <PES2UG24CS547>
Date:   1776543228

    Add farewell

commit 0531aab5497b9a2af16eaf9d4d9ad900f5c7e2985688bf86ac71ce59d0141b86
Author: T S Sathya Kiran <PES2UG24CS547>
Date:   1776543228

    Update file.txt

commit ac2ecbb9ab6bef39aa2ad2d626f26345f1ca0794176a9f1d637ff76a6870ff07
Author: T S Sathya Kiran <PES2UG24CS547>
Date:   1776543228

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
d39ff4c5da7546492023c8f0df5ee97f47b77021c51095805fb735ef3e3ef715

--- Object Store ---
Objects created:
10
.pes/objects/05/31aab5497b9a2af16eaf9d4d9ad900f5c7e2985688bf86ac71ce59d0141b86
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/ac/2ecbb9ab6bef39aa2ad2d626f26345f1ca0794176a9f1d637ff76a6870ff07
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e00de0ccd92
.pes/objects/d8/7733c25d2d137b7574be8c5542b562bf40bafeaa3829f61f75b8d10d5350f9
.pes/objects/d3/9ff4c5da7546492023c8f0df5ee97f47b77021c51095805fb735ef3e3ef715
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
parallels@fedora-linux-42:~/PES2UG24CS547-pes-vcs$
```

Integration test part 2: Full history, reference chain, 10 objects in store, and final "All integration tests completed" banner.

The full end-to-end integration test (`test_sequence.sh`) passes completely:

- Repository initialization creates all required `.pes/` directories and HEAD
- Staging files and checking status works correctly
- Three sequential commits are created with correct hashes and log output
- The full history is displayed correctly newest-to-oldest

- The reference chain (HEAD → refs/heads/main → commit hash) is correct
- 10 objects were created across 3 commits, matching the expected blob + tree + commit structure
- Final line: `=== All integration tests completed ===`

Phase 5: Branching and Checkout (Analysis)

Q5.1 — How would you implement pes checkout ?

A branch in PES-VCS is a file at `.pes/refs/heads/<branch>` containing a commit hash. To implement `pes checkout <branch>`, the following steps are needed:

First, read the target branch file to get its commit hash, then read that commit object to get its root tree hash. Next, walk the working directory and index to detect any uncommitted changes that would conflict with the checkout (see Q5.2). If no conflicts exist, recursively walk the target tree object and write each blob's content to the corresponding file path in the working directory — creating any missing directories as needed, and deleting any tracked files from the current branch that don't exist in the target tree. Finally, update `.pes/HEAD` to contain `ref: refs/heads/<branch>` and reload the index to match the new tree's state.

The complexity comes from several factors. Files that exist in the current branch but not the target must be deleted. Files that exist in both branches but differ must be overwritten. Nested subdirectories must be created or removed as the tree structure changes. The operation must be atomic enough that a failure halfway through doesn't leave the working directory in a broken mixed state — unlike a single-file rename, updating dozens of working directory files is not inherently atomic.

Q5.2 — Detecting dirty working directory conflicts

To detect a conflict before allowing checkout, the following check is performed for each file tracked in the current index:

Compare the file's current on-disk `mtime` and `size` (from `lstat`) against the values stored in the index entry. If they differ, the file has been modified since it was last staged — this is an unstaged modification. Also check whether the index entry's hash matches the HEAD tree's hash for that path — if they differ, the file has been staged but not committed.

If the target branch's tree has a different blob hash for any of these dirty files, checkout must refuse. The check uses only the index (for staged state) and the object store (to look up the target tree's version of each file) — no diff of actual file content is needed, just hash comparison. Files that are dirty but whose path doesn't exist in the target branch are also a conflict, since checkout would need to delete them.

Q5.3 — Detached HEAD state

Detached HEAD occurs when `.pes/HEAD` contains a raw commit hash directly instead of `ref: refs/heads/<branch>`. This happens when you explicitly checkout a specific commit hash rather than a branch name.

In this state, new commits are created and HEAD is updated to point to them directly — but no branch ref file is updated. The commits exist in the object store and are reachable from HEAD, but once you switch away (checkout another branch), HEAD stops pointing to them and they become unreachable from any named reference.

To recover those commits, a user would need to remember (or find in a log) the commit hash they were on before switching. They could then create a new branch pointing to that hash: `pes checkout -b`

`recovery-branch <hash>` — which would create `.pes/refs/heads/recovery-branch` containing that hash, making the commits reachable again. In real Git, `git reflog` keeps a history of where HEAD has pointed, making recovery straightforward. Without a reflog, the only option is searching the object store for commit objects whose hashes you can identify.

Phase 6: Garbage Collection (Analysis)

Q6.1 — Algorithm to find and delete unreachable objects

The algorithm is a mark-and-sweep over the object graph:

Mark phase: Start from every branch ref file in `.pes/refs/heads/` and the current HEAD. For each commit hash found, read the commit object and mark it reachable. Follow its `tree` pointer — read the tree object and mark it reachable, then recursively mark every entry in the tree: blob entries are marked directly, subtree entries are recursed into. Follow the `parent` pointer and repeat the entire process for the parent commit, continuing until a commit with no parent is reached. Use a hash set to track all reachable hashes and avoid revisiting objects.

Sweep phase: Walk every file in `.pes/objects/XX/YYY...` using `find` or `opendir/readdir`. Reconstruct the full hash from the directory name and filename. If the hash is not in the reachable set, delete the file. After deleting all unreachable objects, remove any now-empty shard directories.

For a repository with 100,000 commits and 50 branches, estimating objects to visit: each commit typically touches ~5–20 files (blobs) and ~3–5 tree objects. Assuming an average of 10 unique objects per commit with reasonable deduplication across commits, a rough estimate is 100,000 × 10 = 1,000,000 objects total in the store. The mark phase visits each reachable object once — likely 500,000–800,000 objects across 50 branches with shared history. Total visits: roughly 1–2 million object reads.

Q6.2 — Race condition between GC and concurrent commit

Consider this interleaving:

1. A commit operation calls `object_write(OBJ_BLOB, ...)` for a new file, successfully writing the blob to `.pes/objects/XX/YYY`.
2. GC starts its mark phase. At this exact moment, the new blob exists on disk but the commit object that will reference it hasn't been written yet. The blob has no path from any branch ref — it is unreachable.
3. GC's sweep phase runs and deletes the new blob as unreachable.
4. The commit operation writes the tree object and commit object referencing the now-deleted blob.
5. The commit succeeds (HEAD is updated), but the repository is now corrupt — the blob is referenced by a commit but doesn't exist on disk.

Real Git avoids this with a **grace period**: objects newer than a configurable threshold (default 2 weeks) are never deleted by GC, even if unreachable. Since a commit operation completes in milliseconds, a 2-week grace period makes the race effectively impossible in practice. Git also writes a `COMMIT_EDITMSG` lock file and uses pack-refs locking to serialize GC against concurrent writes. Some implementations additionally use a `.pes/gc.pid` lock file to prevent GC from running at all while any write operation holds the repository lock.